Deep Learning for Credit Card Fraud Detection: Threshold Optimization and Explainable AI Consensus

Shabiba Binta Shahadat
Brac University
Dhaka, Bangladesh
shahadatshabiba@gmail.com

ABSTRACT

Credit card fraud detection has become a major challenge in modern digital financial systems because of the rapid growth of online transactions and the highly imbalanced nature of financial data. This paper presents an end-to-end, explainable deep learning framework that couples a regularised PyTorch Deep Neural Network (DNN) with algorithmic decision-threshold optimization and an Explainable AI (XAI) layer built on SHAP and LIME. Logistic Regression and Random Forest are trained under identical preprocessing as baselines. All models are evaluated in two phases on the Credit Card Fraud Detection Dataset 2023 (568,630 transactions): a balanced 50:50 test set and a simulated production stream with a 99:1 class ratio (56,863 legitimate and 568 fraudulent transactions). The shift in class prior sharply degrades the linear baseline (precision 0.98 to 0.30) and reduces the precision of the Random Forest and the default-threshold DNN to 0.85 and 0.88, respectively. A threshold sweep over the precision–recall curve selects $\theta^* = 0.9740$, which raises DNN precision from 0.88 to 0.94 at unchanged recall of 1.00, lifting F1 from 0.94 to 0.97. SHAP identifies Amount, V12 and V23 as the dominant global drivers, while LIME attributes an individual fraud alert chiefly to low values of V14 and V10. The result is an auditable, adaptive pipeline suited to regulated financial environments.

Index Terms*—Credit Card Fraud Detection, Deep Learning, Explainable AI, SHAP, LIME, Threshold Optimization, Machine Learning, Financial Security.*

I. INTRODUCTION

Among today's financial security concerns, few weigh heavier than credit card fraud. As digital payments expand, deceptive tactics grow in both complexity and volume. Billions of dollars are lost each year by banking institutions, yet the monetary figures capture only part of the damage: customer confidence erodes whenever accounts are compromised, which in turn weakens global transaction networks. The stability of the financial system depends on reliability, and every breach tests that foundation.

The central technical obstacle in automated fraud detection is severe class imbalance. Legitimate transactions vastly outnumber fraudulent ones, and real-life datasets frequently exhibit a hundredfold difference between the two classes. Under such conditions, systems tuned for overall accuracy tend to collapse toward the majority class, and common evaluation practice can hide the fact that a model either misses fraud or floods analysts with false alarms. Accuracy alone is therefore a misleading yardstick; precision, recall and the F1-score of the minority (fraud) class are the quantities that matter operationally.

This work addresses these issues using the Credit Card Fraud Detection Dataset 2023 [4]. We benchmark three model families—Logistic Regression, Random Forest, and a tailored Deep Neural Network—under identical constraints. Rather than relying on the conventional decision boundary of 0.5, we derive an adjusted threshold through *Algorithmic Threshold Optimization* that maximises the F1-score under a skewed class distribution. In parallel, we place Explainable AI at the centre of the design: SHAP and LIME reveal the reasoning behind each fraud alert, so that decisions come with reasons. This matters in finance, where errors cost more than time and where a model must be accepted by the people who act on it, not merely be accurate.

The main contributions of this paper are:

- **Two-phase benchmark.** A like-for-like comparison of Logistic Regression, Random Forest and a PyTorch DNN on a balanced 50:50 test set and on a simulated 99:1 production stream.
- **Prior-shift analysis.** A quantitative account of how the move from balanced to realistic class priors affects each model, showing that precision, not recall, absorbs the damage.
- **Algorithmic threshold calibration.** An F1-maximising search over the precision–recall curve that yields $\theta^* = 0.9740$ and improves DNN precision from 0.88 to 0.94 without sacrificing recall.
- **Global and local explanations.** A SHAP (global) and LIME (local) layer wrapped around the PyTorch model, with a discussion of where the two explanations agree and where they differ.
- **Auditable artefacts.** Saved model checkpoint (weights, configuration and fitted scaler), confusion-matrix heatmaps and full classification reports for every model and threshold condition.

The remainder of the paper is organised as follows. Section II reviews related work. Section III describes the dataset and exploratory analysis. Section IV details the methodology and Section V the system architecture. Section VI presents the experimental protocol, Section VII the results, and Section VIII the explainability analysis. Section IX discusses implications and limitations, and Sections X and XI conclude and outline future work.

II. RELATED WORK AND BACKGROUND

A. Machine Learning for Fraud Detection

Machine learning for transactional fraud detection has been studied extensively, and systematic reviews of the banking sector document a wide range of supervised techniques [2]. Linear models such as Logistic Regression are a common baseline [5], while tree ensembles such as Random Forest exhibit strong performance on tabular data and model non-linear decision boundaries. Deep neural networks have gained prominence for their ability to capture non-linear interactions among latent features [6, 7], and more recent work explores graph neural networks and autoencoders for real-time credit card fraud prevention [1]. Our DNN is deliberately modest in size—two hidden blocks—so that it can be stabilised with Batch Normalization and Dropout, trained quickly, and explained with model-agnostic tools.

B. Evaluation Under Class Imbalance

Realistic modelling of credit card fraud detection must account for the strong imbalance between classes [3]. When the class prior at deployment differs from that at training or test time—a *prior probability shift*—metrics that depend on the class mix, notably precision, can change dramatically even if the classifier itself is unchanged. Accuracy is misleading in this setting, so we build on the relationship between precision–recall and ROC curves [8] and use the precision–recall curve as the basis for selecting a decision threshold.

C. Explainable AI in Financial Decision-Making

Two persistent gaps motivate this work. First, *imbalance sensitivity*: models trained or evaluated under balanced assumptions can suffer steep precision deterioration when subjected to realistic, highly imbalanced streams. Second, *opacity and regulation*: regulatory frameworks such as the GDPR require financial institutions to provide algorithmic transparency, yet deep networks offer no built-in interpretability. Post-hoc explainers address this second gap. SHAP assigns each feature a Shapley-value contribution grounded in cooperative game theory [9], while LIME fits a local surrogate model around a single prediction [10]. Their use in adaptive real-time fraud detection has been examined in the literature [11]. This paper bridges both gaps by pairing DNN classification with an empirical threshold search and with dual-level (global and local) interpretability.

III. DATASET AND EXPLORATORY DATA ANALYSIS

A. Dataset Description

We use the publicly available Credit Card Fraud Detection Dataset 2023 [4], which contains 568,630 anonymised transactions made by European cardholders. Each record carries 28 PCA-transformed numerical features, V1–V28, produced by the dataset's authors to protect confidential information, together with the transaction `Amount` and the binary target `Class` (0 = legitimate, 1 = fraudulent). A surrogate identifier column, `id`, has no predictive value and is removed. The dataset is balanced between the two classes, which we refer to as the 50:50 or "synthetically balanced" setting; this is precisely why a separate, realistic imbalanced evaluation is needed (Section IV-D). Fig. 1 summarises the key figures of the experimental setup.

TABLE I
MODEL CONFIGURATIONS

Model	Key settings
Logistic Regression	L-BFGS solver, max. 1000 iterations; 29 features (RobustScaled Amount + V1–V28); no further scaling.
Random Forest	100 decision trees, maximum depth 10 (implicit regularisation); same 29 features.
DNN	StandardScaler fitted on training data only; 29–64–32–1; ReLU, BatchNorm1d, Dropout 0.3 per block; Sigmoid output; Adam (10^{-3}); batch 128; BCE loss; $\leq$50 epochs; early stopping (patience 4) on a validation set of 20% of the training data.

B. Exploratory Data Analysis

Exploratory analysis was carried out to understand the structure of the data and the behaviour of transactions before modelling. It comprised:

- **Correlation analysis** via a heatmap of the feature relationships, which highlighted several features strongly related to fraud and thus helped identify influential variables.
- **Transaction-amount analysis** using box plots, which showed noticeable differences between fraudulent and legitimate transactions; fraudulent ones often displayed abnormal patterns of spending.
- **Feature-distribution analysis** using kernel density estimate (KDE) curves of the PCA-transformed variables, several of which showed clearly distinct distributions for the two classes.
- **Pair plots and a two-dimensional t-SNE projection** of the feature space to inspect class separability visually.

Taken together, these analyses gave insight into class structure, transaction patterns and feature relationships, and they supported the design of the proposed framework.

IV. METHODOLOGY

A. Data Preprocessing

After removing the `id` column, the `Amount` attribute is scaled with scikit-learn's `RobustScaler`, which is robust to the outliers that are common in fraud data:

$$X_{\text{scaled}} = \frac{X - Q_2(X)}{Q_3(X) - Q_1(X)}, \tag{1}$$

where Q_1, Q_2 and Q_3 are the first quartile, median and third quartile of the training distribution. The scaled amount is concatenated with V1–V28, which are already standardised through PCA by the dataset's authors, forming the 29-dimensional feature vector used by every model. The data are then partitioned into 80% for training and 20% for testing using stratified sampling with a fixed random seed of 42, so that the class distribution is identical in both partitions. No resampling of the training data is applied; because the training data are already balanced, techniques such as SMOTE are left to future work.

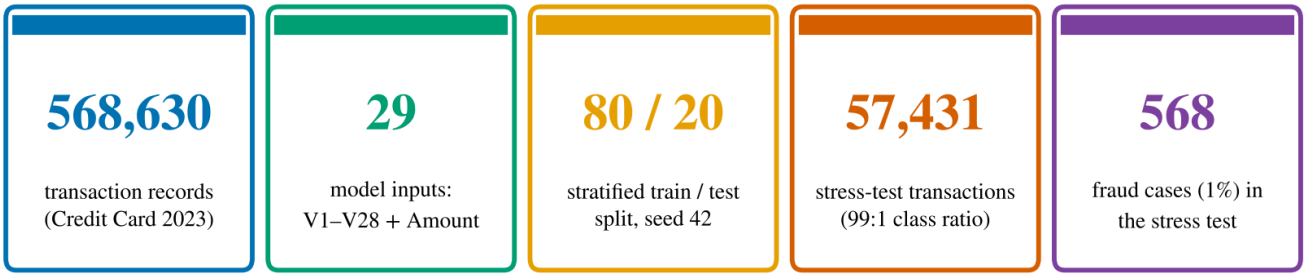


Fig. 1. Experimental setup at a glance: dataset size, model inputs, data partition and the composition of the 99:1 stress-test stream.

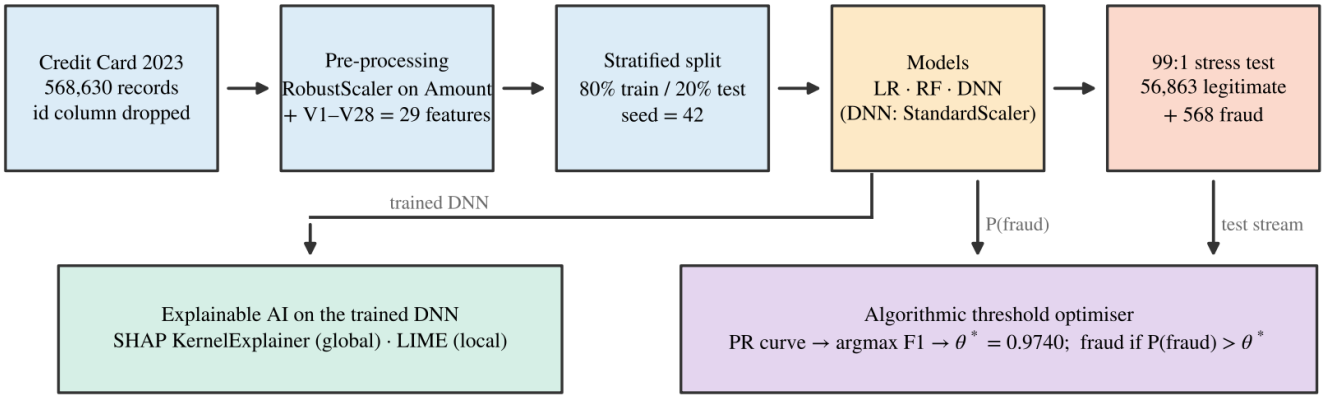


Fig. 2. End-to-end pipeline: preprocessing, stratified split, three model families, the 99:1 stress test, algorithmic threshold optimisation, and the SHAP/LIME explanation layer applied to the trained DNN.

B. Model Development

1) Logistic Regression. Logistic Regression serves as the linear baseline [5]. It is trained with the L-BFGS solver for at most 1000 iterations, stopping automatically at convergence, on the 29-feature set with no additional scaling.

2) Random Forest. The Random Forest baseline uses 100 decision trees with a maximum depth of 10, which acts as an implicit regularizer and limits overfitting. It uses the same preprocessed features as Logistic Regression.

3) Deep Neural Network. The DNN is the main model [6]. A second scaler, a `StandardScaler`, is fitted on the training set only and applied to all 29 features (the RobustScaled Amount and V1–V28), so the network receives fully standardised, zero-mean, unit-variance inputs. This matters because gradient-descent-based learners are sensitive to differences in feature magnitude. The architecture is a two-block fully connected network, detailed in Section V. Table I summarises the configuration of all three models.

C. DNN Training Procedure

The DNN is trained with the Adam optimizer at a learning rate of 10^{-3} on mini-batches of 128 samples, minimising the Binary Cross-Entropy (BCE) loss

$$\mathscr{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\Big], \qquad (2)$$

where $\hat{y}_i$ is the predicted fraud probability. Training runs for up to 50 epochs on 80% of the training data, with the remaining 20% held out as a validation set. Early stopping halts training when the validation loss fails to improve for four consecutive epochs, and the weights with the lowest validation loss are restored. The trained model, its configuration and the fitted StandardScaler are saved in a single checkpoint for reproducible reuse in the subsequent evaluation and XAI stages. For every model, a confusion-matrix heatmap is saved as a PNG and a full classification report as a text file, providing a complete audit trail of the evaluation.

D. Real-World Imbalanced Simulation

To assess the models under conditions closer to production, we construct an imbalanced test stream from the test split. All legitimate test transactions are retained (56,863), while fraudulent transactions are sub-sampled to roughly 1% of the stream, giving 568 fraud cases. The resulting set of 57,431 transactions has a class ratio of approximately 99:1, close to the fraud rates seen in production [1]. This stream is used to evaluate the DNN with the fixed threshold of 0.5 and with the algorithmically optimised threshold; confusion-matrix heatmaps and classification reports are stored for both conditions. Logistic Regression and Random Forest are evaluated on the same stream.

E. Algorithmic Threshold Optimization

Binary classifiers conventionally apply a static decision boundary of $\theta = 0.5$. When a model trained on balanced (50:50) data is deployed in a severely imbalanced (e.g., 99:1) environment, this

static rule produces a high volume of false positives and therefore operational friction. To resolve this, the raw probabilistic outputs of the DNN are routed into a programmatic optimisation module. Instead of heuristic guessing, the module computes the continuous precision–recall curve across the entire distribution of predicted probabilities [8]. For every candidate threshold t, the F1-score—the harmonic mean of precision $P(t)$ and recall $R(t)$—is

$$F_1(t) = \frac{2P(t)R(t)}{P(t)+R(t)+\varepsilon}, \qquad \varepsilon = 10^{-10}, \tag{3}$$

where ε prevents division by zero. The optimal decision boundary is the maximiser

$$\theta^* = \arg\max_t F_1(t), \tag{4}$$

and the final inference rule bypasses the default classifier: *flag as fraudulent if* $P(\text{fraud}) > \theta^*$, *otherwise legitimate*. Because θ^* is derived from the observed score distribution rather than fixed a priori, the calibration adapts to the class distribution of the deployment environment.

F. Explainable AI Pipeline

Two complementary XAI methods make the trained DNN interpretable. SHAP and LIME operate on NumPy matrices whereas the model expects PyTorch tensors, so a wrapper function was written: it accepts a NumPy matrix, converts it to a tensor, runs a forward pass through the DNN and returns the class probabilities $[P(\text{legitimate}), P(\text{fraud})]$. This wrapper bridges the two frameworks and serves as the prediction function for both explainers.

1) Global interpretability (SHAP). A SHAP `KernelExplainer` is built with 100 background samples and applied to 200 test samples. The mean absolute SHAP value of each feature is used to produce a ranked summary of average impact on the model output, i.e. a population-level view of which features drive the fraud prediction [9].

2) Local interpretability (LIME). A `LimeTabularExplainer` fitted on the training distribution explains individual decisions. From an evaluation set of 5,000 samples, the first fraud sample (sample #0) is explained, and the six most influential features are reported [10]. SHAP thus yields feature importance across the population, whereas LIME gives a local, instance-level explanation of a single fraud decision.

V. System Architecture

The challenges of the PCA-transformed feature space and of real-world class imbalance are handled by a two-stage computational pipeline: a custom DNN that learns latent feature interactions, followed by an algorithmic threshold optimiser that calibrates the decision boundary. The end-to-end pipeline, including the XAI layer, is shown in Fig. 2.

A. Deep Neural Network Architecture

The 29-dimensional input vector is standardised and fed into a feed-forward PyTorch network composed of two heavily regularised hidden blocks (Fig. 3).

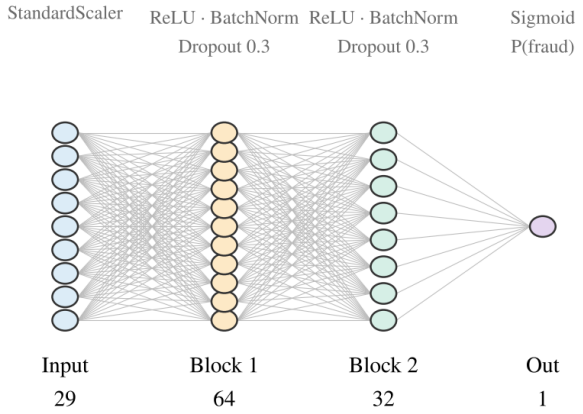


Fig. 3. DNN architecture (29–64–32–1). Hidden blocks apply ReLU, Batch Normalization and Dropout ($p = 0.3$); the output neuron applies a Sigmoid. Nodes are subsampled for legibility.

1) Hidden block 1. The input layer projects the 29 features into a 64-dimensional fully connected space. Non-linearity is introduced by the Rectified Linear Unit,

$$f(x) = \max(0, x). \tag{5}$$

To accelerate convergence and mitigate internal covariate shift, 1-D Batch Normalization follows. A Dropout layer with probability $p = 0.3$ then guards against overfitting to the synthetic equilibrium of the balanced training data.

2) Hidden block 2. The network compresses the learned patterns into a denser 32-dimensional representation through an identical sequence of ReLU activation, Batch Normalization and Dropout with $p = 0.3$.

3) Output layer. The final layer reduces the 32-dimensional vector to a single scalar. A Sigmoid activation bounds this value to the probabilistic range, yielding the raw fraud probability

$$P(\text{fraud}) = \sigma(z) \in [0, 1]. \tag{6}$$

B. Threshold Optimizer and Decision Rule

The probability scores are passed to the threshold optimiser, which sweeps the precision–recall curve, computes $F_1(t)$ at every candidate threshold and selects θ^* by taking the argmax (Section IV-F). During inference, a transaction is flagged as fraudulent if $P(\text{fraud}) > \theta^*$. This calibration is purely post-hoc: it requires no retraining of the network, and it can be re-run whenever the class distribution of the deployment environment changes.

VI. Experimental Protocol

A. Evaluation Phases

Testing was conducted in two distinct phases. **Phase I** assesses the models on the synthetically balanced (50:50) test split. **Phase II** subjects them to a simulated real-world environment with a 99:1 class imbalance (the prior probability shift) comprising 56,863 legitimate and 568 fraudulent transactions.

TABLE II
PERFORMANCE METRICS ACROSS EQUILIBRIUM AND IMBALANCED DATASETS (FRAUD CLASS)

Model	Test distribution	Prec.	Recall	F1
Logistic Regression	50:50 (standard)	0.98	0.95	0.96
Random Forest	50:50 (standard)	1.00	0.97	0.99
Deep Neural Network	50:50 (standard)	1.00	1.00	1.00
Logistic Regression	99:1 (imbalanced)	0.30	0.96	0.46
Random Forest	99:1 (imbalanced)	0.85	0.98	0.91
DNN ($\theta = 0.50$)	99:1 (imbalanced)	0.88	1.00	0.94
DNN ($\theta^* = 0.9740$)	**99:1 (imbalanced)**	**0.94**	**1.00**	**0.97**

B. Metrics

Raw accuracy is fundamentally misleading in imbalanced settings, so evaluation focuses on the fraud class using

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}, \qquad (7)$$

where TP, FP and FN are the true positives, false positives and false negatives of the fraud class. Precision measures how many fraud alerts are correct (and hence the burden of false alarms), recall measures how many fraudulent transactions are intercepted, and F_1 balances the two.

C. Configurations Compared

Seven configurations are reported: the three models on the balanced split, and on the imbalanced stream Logistic Regression, Random Forest, the DNN at the default threshold $\theta = 0.50$, and the DNN at the optimised threshold θ^*. All models share the same preprocessing, the same stratified split and the same random seed (42).

VII. RESULTS AND ANALYSIS

A. Phase I: Performance on Synthetic Equilibrium

Under balanced conditions the PCA-transformed feature space exhibits high linear separability (Table II, Fig. 4a). Logistic Regression already attains a robust F1-score of 0.96 (precision 0.98, recall 0.95). The Random Forest exploits non-linear boundaries to reach a near-perfect F1-score of 0.99, with a slight recall deficit of 0.97 that indicates a small margin of false negatives. The DNN achieves precision and recall of 1.00 on this set. This confirms that a deep architecture, stabilised with global scaling and Batch Normalization, can map the continuous, synthetic manifolds generated by the dataset's authors essentially without error. Because these results are near ceiling, the balanced test set alone cannot discriminate between the strongest models, which motivates Phase II.

B. Phase II: Impact of Prior Probability Shift

Introducing the 99:1 prior shift induces a catastrophic loss of precision in the linear baseline (Fig. 4b, Fig. 5). Logistic Regression precision collapses from 0.98 to 0.30: more than 70% of its fraud alerts are incorrect, even though its recall (0.96) is intact. The Random Forest proves more resilient, retaining recall of 0.98, but its precision falls to 0.85, highlighting the limits of a static decision rule on heavily skewed data. At the default boundary $\theta = 0.50$, the DNN keeps a perfect recall of 1.00 and intercepts all 568 fraudulent transactions, yet its precision drops to 0.88, reflecting operational false positives triggered by the shift.

TABLE III
EFFECT OF PRIOR PROBABILITY SHIFT (50:50 → 99:1)

Model	ΔPrecision	ΔRecall	ΔF1
Logistic Regression	−0.68	+0.01	−0.50
Random Forest	−0.15	+0.01	−0.08
DNN ($\theta = 0.50$)	−0.12	0.00	−0.06
DNN (θ^*)	−0.06	0.00	−0.03

A notable pattern is that recall is essentially unaffected by the shift (LR 0.95→0.96, RF 0.97→0.98, DNN 1.00→1.00), whereas precision falls for every model. This is expected: recall is computed only over fraudulent transactions, whereas precision depends on the mix of classes. Table III quantifies the precision loss. For the same fraud-class behaviour, every additional legitimate transaction that scores above the threshold adds a false positive, so models with weaker separation of the two classes pay a disproportionately higher price.

C. DNN Algorithmic Threshold Calibration

Fig. 6 shows the F1-score as a function of the decision threshold. The curve rises steeply over the lowest thresholds, passes roughly 0.9 near $t = 0.2$, and then climbs gradually toward a maximum close to 0.97 before collapsing sharply as t approaches 1.0. The algorithm identifies the optimal boundary at $\theta^* = 0.9740$. The broad plateau suggests that the F1-score is not hypersensitive to the exact threshold across a wide mid-range, but the sharp drop beyond the optimum warns against pushing the threshold arbitrarily close to 1.

Applying this calibrated threshold yields the results shown in Table II and Fig. 7. The optimised DNN raises minority-class precision from 0.88 to 0.94 while recall remains a flawless 1.00, lifting F1 from 0.94 to 0.97. Since recall is unchanged, the fraud interceptions are identical and the entire gain comes from suppressing false alarms: the share of incorrect alerts (one minus precision) falls from 0.12 to 0.06, i.e. roughly halves. This shows that calibrating the probabilistic output of a DNN is a more effective way of reducing false positives than relying on a static threshold, and the optimised DNN outperforms the Random Forest (F1 0.91) and Logistic Regression (F1 0.46) on the same stream.

D. Comparative Discussion

Three observations stand out. First, the ranking of models is stable across phases (DNN $\geq$ Random Forest $>$ Logistic Regression), but the *gaps* widen dramatically under realistic priors: the F1 difference between the DNN and Logistic Regression grows from 0.04 in Phase I to 0.51 in Phase II at the optimised threshold. Second, traditional methods retain value: Logistic Regression is a solid baseline on balanced data and the Random Forest reaches F1 of 0.99 there. Third, the cheapest improvement in the whole pipeline is the threshold: it requires no retraining, no resampling and no architectural change.

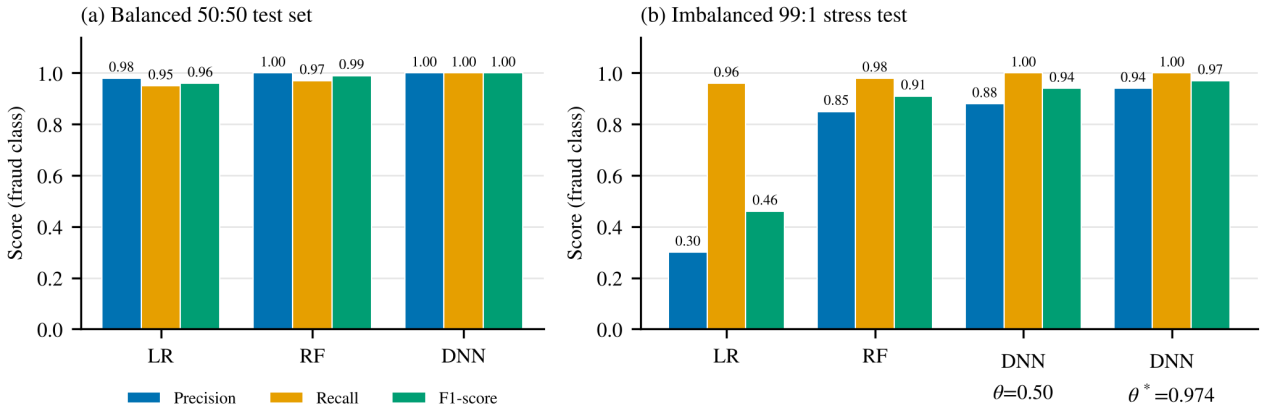


Fig. 4. Precision, recall and F1-score of the fraud class for each model on (a) the balanced 50:50 test set and (b) the 99:1 stress test. The class-prior shift mainly erodes precision.

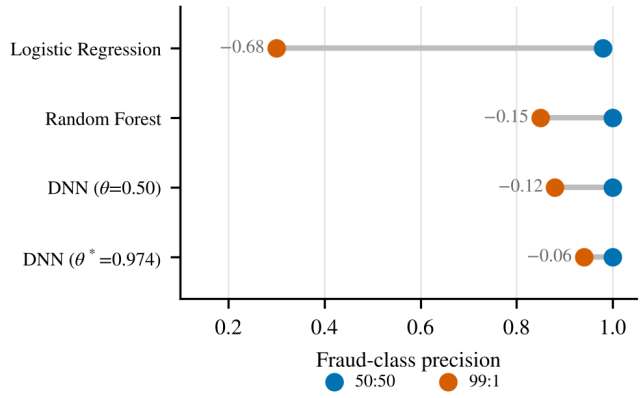


Fig. 5. Fraud-class precision on the balanced (blue) and 99:1 (red) test sets. Labels show the absolute precision lost under prior shift.

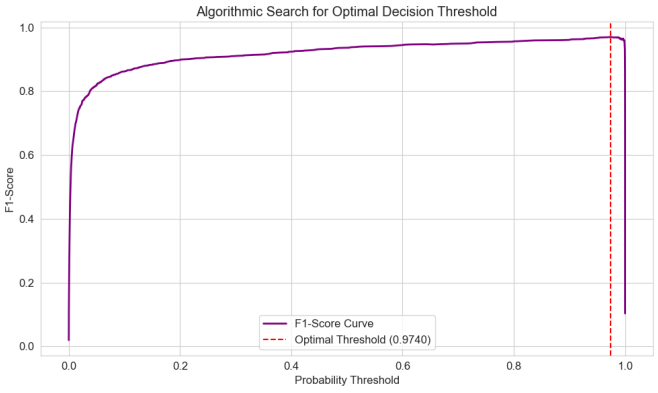


Fig. 6. Algorithmic search for the optimal decision threshold: F1-score versus probability threshold; the dashed line marks $\theta^* = 0.9740$.

VIII. EXPLAINABLE AI ANALYSIS

While the calibrated DNN shows exceptional predictive performance, its "black-box" character poses a significant obstacle to real-world financial deployment, where regulatory frameworks such as the GDPR call for algorithmic transparency. To retain

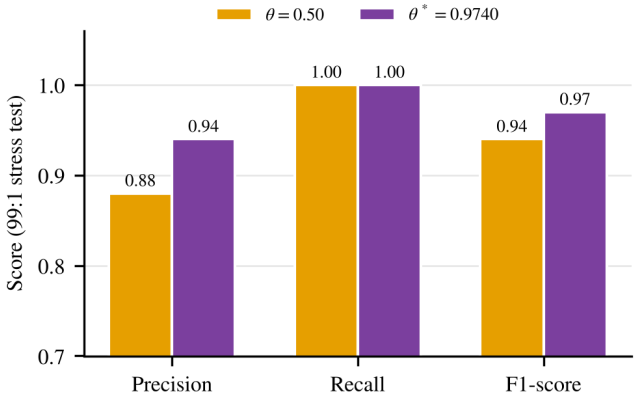


Fig. 7. Effect of the optimised threshold on the DNN in the 99:1 stress test: precision improves by 0.06 and F1 by 0.03 at unchanged recall.

TABLE IV
SUMMARY OF THE XAI CONFIGURATION

Method	Scope	Setting and output
SHAP	Global	KernelExplainer; 100 background and 200 test samples; ranking by mean |SHAP value|.
LIME	Local	LimeTabularExplainer on the training distribution; first fraud sample of a 5,000-sample evaluation set; top six features.

the network's expressive power while meeting that need, a cross-methodological XAI framework was applied to the PyTorch model using both a global and a local algorithm (Table IV).

A. Global Interpretability via SHAP

SHAP calculates the marginal contribution of each feature, grounded in cooperative game theory [9]. The mean absolute Shapley values in Fig. 8 show that the transaction `Amount` is the single strongest global driver, with an importance more than three times that of the next feature. It is followed by the PCA components V12 and V23, and then by V4, V10, V3, V11, V7, V2 and V14. The remaining displayed components (V17–V22 and V24, V26–V28) contribute almost nothing. The network

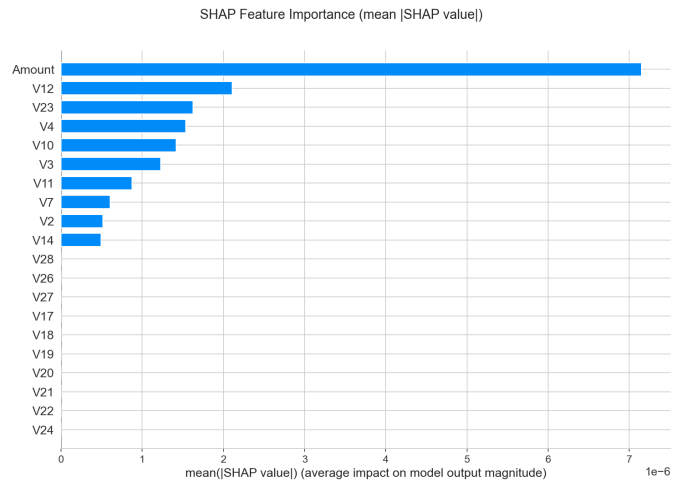


Fig. 8. SHAP global feature importance (mean absolute SHAP value) for the fraud prediction of the DNN.

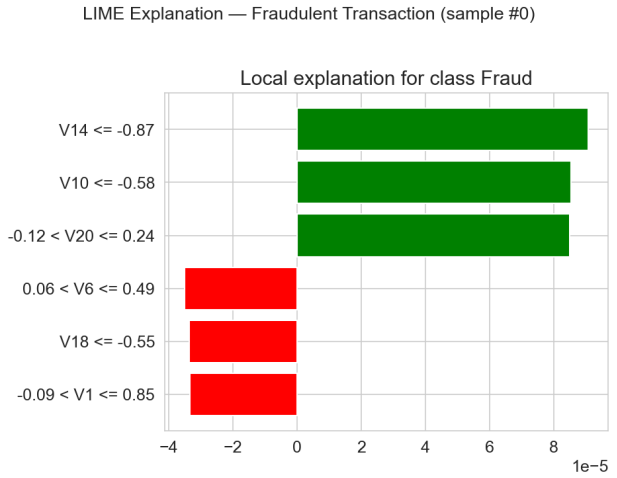


Fig. 9. LIME local explanation for a fraudulent transaction (sample #0). Green bars support the fraud class; red bars oppose it.

therefore concentrates its decision weight on a small subset of the 29 inputs to establish its baseline separation between legitimate and fraudulent transactions. The absolute SHAP magnitudes are small (order 10^{-6}, as the axis in Fig. 8 indicates), so it is the relative ranking, rather than the scale, that carries the interpretation.

B. Local Interpretability via LIME

To validate behaviour at the level of an individual transaction, LIME perturbs the feature space around a single instance and fits a local surrogate model [10]. Fig. 9 shows the explanation for a verified fraudulent transaction (sample #0). The strongest positive contributors (green bars), which push the model toward a fraud classification, are low values of V14 (≤ -0.87) and V10 (≤ -0.58), together with V20 lying in the interval $(-0.12, 0.24]$. Three conditions work against the fraud decision (red bars): V6 in $(0.06, 0.49]$, V18 ≤ -0.55 and V1 in $(-0.09, 0.85]$.

C. Consensus Between Global and Local Explanations

Comparing the two explainers shows both agreement and complementarity. V10 and V14 appear in the SHAP ranking and are also leading local contributors in the LIME explanation of the fraud alert; in LIME their *low* values push toward fraud. This is consistent with the correlation structure observed during exploratory analysis, where several PCA components were found to be related to fraud, and it suggests that the DNN has captured meaningful, class-relevant structure rather than arbitrary patterns. The two explanations are not identical, however. `Amount` and V12, the top global drivers, are absent from the six features LIME reports for sample #0, whereas V20, V6, V18 and V1 appear locally but contribute almost nothing globally. This is the expected division of labour between the methods: SHAP summarises average behaviour across the population, while LIME exposes the particular conditions that tipped one decision. We therefore regard the explainers as complementary evidence, and confidence rises where they overlap (V10, V14) rather than from either method alone. Together they let an analyst answer both "which features does this model rely on?" and "why was *this* transaction flagged?".

IX. Discussion

A. Practical Implications

The results carry three practical lessons for deployment. First, validation on balanced data is insufficient: a model that looks flawless under a 50:50 split can generate a substantial share of false alerts when the prior shifts to 99:1, and the linear baseline demonstrates how severe this can be (precision 0.30). Second, threshold calibration is an inexpensive, retraining-free lever that directly addresses the operational cost of false alarms, since it halves the share of incorrect alerts for the DNN at no cost in recall. Third, pairing predictions with explanations means that automated fraud interventions can be justified to customers, auditors and regulators, while the saved checkpoints, reports and confusion matrices make each evaluation step auditable.

B. Limitations and Threats to Validity

Several caveats should be kept in mind when interpreting the results.

- **Synthetic benchmark.** The dataset is balanced by construction, and the near-perfect Phase I scores of the DNN reflect how cleanly its authors' synthetic manifolds can be separated. Absolute performance on live transaction streams may differ.
- **Simulated prior shift.** The 99:1 stream is derived by subsampling fraud cases from the test split, not from a temporally ordered production log; concept drift and behavioural change over time are therefore not captured.
- **Threshold selection data.** The threshold is derived from the precision–recall curve of the predicted probabilities and applied to the imbalanced stream. For production use it should be calibrated on a held-out validation stream separate from the final evaluation data to avoid optimistic estimates.
- **Single split and seed.** Results are reported for one stratified split (seed 42) without confidence intervals, so small differences (for example F1 0.94 versus 0.97) should be read as indicative.
- **Explainer scope.** The SHAP analysis uses 100 background and 200 test samples, and the LIME explanation covers a

single fraud instance; broader sampling would strengthen the explanatory conclusions.

X. CONCLUSION

This paper presented an end-to-end deep learning framework for credit card fraud detection that combines a regularised PyTorch DNN, algorithmic decision-threshold optimisation and a two-level explainability layer. On the balanced test set the DNN reached precision and recall of 1.00, while Logistic Regression (F1 0.96) and Random Forest (F1 0.99) confirmed the continued relevance of traditional methods. Under a realistic 99:1 prior shift the picture changed: Logistic Regression precision collapsed to 0.30, the Random Forest dropped to 0.85, and the DNN at the default threshold to 0.88, with an F1 of 0.94. An F1-maximising search over the precision–recall curve selected $\theta^* = 0.9740$, which raised precision to 0.94 at unchanged recall of 1.00 and F1 to 0.97, roughly halving the share of false alerts. SHAP highlighted Amount, V12 and V23 as the dominant global drivers, and LIME showed that low values of V14 and V10 triggered the alert for an individual fraudulent transaction, with V10 and V14 recurring in both views. Interpretability thus emerged alongside performance rather than at its expense, a balance that becomes critical when systems move from testing into live environments. The complete implementation, covering the training pipeline and the SHAP and LIME explanation scripts, is available at `https://github.com/shabibabintashahadat/credit-card-fraud-detection-xai`.

XI. FUTURE WORK

Although the outcomes are positive, several directions could strengthen these findings.

- **More aggressive imbalance handling.** Techniques such as SMOTE, cost-sensitive learning that weights rare cases more heavily, and focal loss, which focuses training on harder instances, could reduce the need for manual rebalancing.
- **Advanced architectures.** Transformers for tabular data and Graph Neural Networks, which model relationships explicitly, may uncover structure beyond the reach of current feed-forward networks.
- **Online and streaming detection.** Integrating streaming data flows would allow continuous model updates and adaptive monitoring rather than static analysis.
- **Temporal and behavioural analysis.** Modelling sequences of actions over time could reveal sudden shifts in activity that static checks miss.
- **Unified XAI consensus.** Combining the outputs of SHAP, LIME and similar techniques into a single consensus score would reduce reliance on any one explainer, since conclusions hold stronger when several tools converge.
- **Deployment and monitoring.** Packaging the model as an API with drift detection would support continuous evaluation as fraud behaviour evolves.

REFERENCES

[1] F. K. Alarfaj and S. Shahzadi, "Enhancing fraud detection in banking with deep learning: Graph neural networks and autoencoders for real-time credit card fraud prevention," *IEEE Access*, vol. 13, pp. 20633–20646, 2025.

[2] O. Jeri-Alvarado, C. Espinoza, J. Gamboa-Cruzado, M. L. Morales, V. Ataupillco-Vera, O. Chavez-Chavez, C. A. T. Romero, and F. A. Castillo-Velázquez, "Financial fraud detection in the banking sector using machine learning: An exhaustive systematic review," *Computación y Sistemas*, vol. 29, no. 3, pp. 1701–1721, 2025.

[3] A. Dal Pozzolo *et al.*, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018.

[4] N. Giriyewithana, "Credit Card Fraud Detection Dataset 2023," Kaggle, 2023.

[5] E. Y. Boateng and D. A. Abaye, "A review of the logistic regression model with emphasis on medical research," *J. Data Anal. Inf. Process.*, vol. 7, no. 4, pp. 190–207, 2019.

[6] T. T. Nguyen, H. Tahir, M. Abdelrazek, and A. Babar, "Deep learning methods for credit card fraud detection," arXiv preprint arXiv:2012.03754, 2020.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[8] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.

[10] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.

[11] G. Alessi and M. Fugini, "Adaptive real-time financial fraud detection with explainable AI tools," *Digital Threats: Research and Practice*, 2026.